

square fit is that the experimenter, in a “fit” of conservatism, has *overestimated* his or her measurement errors. Very rarely, too good a chi-square signals actual fraud, data that have been “fudged” to fit the model.

A rule of thumb is that a “typical” value of χ^2 for a “moderately” good fit is $\chi^2 \approx \nu$. More precise is the statement that the χ^2 statistic has a mean ν and a standard deviation $\sqrt{2\nu}$ and, asymptotically for large ν , becomes normally distributed.

In some cases the uncertainties associated with a set of measurements are not known in advance, and considerations related to χ^2 fitting are used to derive a value for σ . If we assume that all measurements have the same standard deviation, $\sigma_i = \sigma$, and that the model does fit well, then we can proceed by first assigning an arbitrary constant σ to all points, next fitting for the model parameters by minimizing χ^2 , and finally recomputing

$$\sigma^2 = \sum_{i=0}^{N-1} [y_i - y(x_i)]^2 / (N - M) \quad (15.1.7)$$

Obviously, this approach prohibits an independent assessment of goodness-of-fit, a fact occasionally missed by its adherents. When, however, the measurement error is not known, this approach at least allows *some* kind of error bar to be assigned to the points.

If we take the derivative of equation (15.1.6) with respect to the parameters a_k , we obtain equations that must hold at the chi-square minimum:

$$0 = \sum_{i=0}^{N-1} \left(\frac{y_i - y(x_i)}{\sigma_i^2} \right) \left(\frac{\partial y(x_i | \dots a_k \dots)}{\partial a_k} \right) \quad k = 0, \dots, M - 1 \quad (15.1.8)$$

Equation (15.1.8) is, in general, a set of M nonlinear equations for the M unknown a_k . Various of the procedures described subsequently in this chapter derive from (15.1.8) and its specializations.

CITED REFERENCES AND FURTHER READING:

- Lupton, R. 1993, *Statistics in Theory and Practice* (Princeton, NJ: Princeton University Press), Chapters 10–11.[1]
 Devore, J.L. 2003, *Probability and Statistics for Engineering and the Sciences*, 6th ed. (Belmont, CA: Duxbury Press), Chapter 6.
 Gelman, A., Carlin, J.B., Stern, H.S., and Rubin, D.B. 2004, *Bayesian Data Analysis*, 2nd ed. (Boca Raton, FL: Chapman & Hall/CRC), Chapter 8.

15.2 Fitting Data to a Straight Line

A concrete example will make the considerations of the previous section more meaningful. We consider the problem of fitting a set of N data points (x_i, y_i) to a straight-line model

$$y(x) = y(x|a, b) = a + bx \quad (15.2.1)$$

This problem is often called *linear regression*, a terminology that originated, long ago, in the social sciences. We assume that the uncertainty σ_i associated with each

measurement y_i is known, and that the x_i 's (values of the dependent variable) are known exactly.

To measure how well the model agrees with the data, we use the chi-square merit function (15.1.6), which in this case is

$$\chi^2(a, b) = \sum_{i=0}^{N-1} \left(\frac{y_i - a - bx_i}{\sigma_i} \right)^2 \quad (15.2.2)$$

If the measurement errors are normally distributed, then this merit function will give maximum likelihood parameter estimations of a and b ; if the errors are not normally distributed, then the estimations are not maximum likelihood but may still be useful in a practical sense. In §15.7, we will treat the case where outlier points are so numerous as to render the χ^2 merit function useless.

Equation (15.2.2) is minimized to determine a and b . At its minimum, derivatives of $\chi^2(a, b)$ with respect to a, b vanish:

$$\begin{aligned} 0 = \frac{\partial \chi^2}{\partial a} &= -2 \sum_{i=0}^{N-1} \frac{y_i - a - bx_i}{\sigma_i^2} \\ 0 = \frac{\partial \chi^2}{\partial b} &= -2 \sum_{i=0}^{N-1} \frac{x_i(y_i - a - bx_i)}{\sigma_i^2} \end{aligned} \quad (15.2.3)$$

These conditions can be rewritten in a convenient form if we define the following sums:

$$\begin{aligned} S &\equiv \sum_{i=0}^{N-1} \frac{1}{\sigma_i^2} & S_x &\equiv \sum_{i=0}^{N-1} \frac{x_i}{\sigma_i^2} & S_y &\equiv \sum_{i=0}^{N-1} \frac{y_i}{\sigma_i^2} \\ S_{xx} &\equiv \sum_{i=0}^{N-1} \frac{x_i^2}{\sigma_i^2} & S_{xy} &\equiv \sum_{i=0}^{N-1} \frac{x_i y_i}{\sigma_i^2} \end{aligned} \quad (15.2.4)$$

With these definitions (15.2.3) becomes

$$\begin{aligned} aS + bS_x &= S_y \\ aS_x + bS_{xx} &= S_{xy} \end{aligned} \quad (15.2.5)$$

The solution of these two equations in two unknowns is calculated as

$$\begin{aligned} \Delta &\equiv SS_{xx} - (S_x)^2 \\ a &= \frac{S_{xx}S_y - S_xS_{xy}}{\Delta} \\ b &= \frac{SS_{xy} - S_xS_y}{\Delta} \end{aligned} \quad (15.2.6)$$

Equation (15.2.6) gives the solution for the best-fit model parameters a and b .

We are not done, however. We must estimate the probable uncertainties in the estimates of a and b , since obviously the measurement errors in the data must introduce some uncertainty in the determination of those parameters. If the data are

independent, then each contributes its own bit of uncertainty to the parameters. Consideration of propagation of errors shows that the variance σ_f^2 in the value of any function will be

$$\sigma_f^2 = \sum_{i=0}^{N-1} \sigma_i^2 \left(\frac{\partial f}{\partial y_i} \right)^2 \quad (15.2.7)$$

For the straight line, the derivatives of a and b with respect to y_i can be directly evaluated from the solution:

$$\begin{aligned} \frac{\partial a}{\partial y_i} &= \frac{S_{xx} - S_x x_i}{\sigma_i^2 \Delta} \\ \frac{\partial b}{\partial y_i} &= \frac{S x_i - S_x}{\sigma_i^2 \Delta} \end{aligned} \quad (15.2.8)$$

Summing over the points as in (15.2.7), we get

$$\begin{aligned} \sigma_a^2 &= S_{xx} / \Delta \\ \sigma_b^2 &= S / \Delta \end{aligned} \quad (15.2.9)$$

which are the variances in the estimates of a and b , respectively. We will see in §15.6 that an additional number is also needed to characterize properly the probable uncertainty of the parameter estimation. That number is the *covariance* of a and b , and (as we will see below) is given by

$$\text{Cov}(a, b) = -S_x / \Delta \quad (15.2.10)$$

The coefficient of correlation between the uncertainty in a and the uncertainty in b , which is a number between -1 and 1 , follows from (15.2.10) (compare equation 14.5.1),

$$r_{ab} = \frac{-S_x}{\sqrt{S S_{xx}}} \quad (15.2.11)$$

A positive value of r_{ab} indicates that the errors in a and b are likely to have the same sign, while a negative value indicates the errors are anticorrelated, likely to have opposite signs.

We are *still* not done. We must estimate the goodness-of-fit of the data to the model. Absent this estimate, we have not the slightest indication that the parameters a and b in the model have any meaning at all! The probability Q that a value of chi-square as *poor* as the value (15.2.2) should occur by chance is

$$Q = 1 - \text{Chisqdist}(N - 2).\text{invcdf}(\chi^2) \quad (15.2.12)$$

Here `Chisqdist` is our object embodying the chi-square distribution function (see §6.14.8) and `invcdf` is its inverse cumulative distribution function. If Q is larger than, say, 0.1 , then the goodness-of-fit is believable. If it is larger than, say, 0.001 , then the fit *may* be acceptable if the errors are nonnormal or have been moderately underestimated. If Q is less than 0.001 , then the model and/or estimation procedure can rightly be called into question. In this latter case, turn to §15.7 to proceed further.

If you do not know the individual measurement errors of the points σ_i , and are proceeding (dangerously) to use equation (15.1.7) for estimating these errors, then

here is the procedure for estimating the probable uncertainties of the parameters a and b : Set $\sigma_i \equiv 1$ in all equations through (15.2.6), and multiply σ_a and σ_b , as obtained from equation (15.2.9), by the additional factor $\sqrt{\chi^2/(N-2)}$, where χ^2 is computed by (15.2.2) using the fitted parameters a and b . As discussed above, this procedure is equivalent to *assuming* a good fit, so you get no independent goodness-of-fit probability Q .

In §14.5 we promised a relation between the linear correlation coefficient r (equation 14.5.1) and a goodness-of-fit measure, χ^2 (equation 15.2.2). For unweighted data (all $\sigma_i = 1$), that relation is

$$\chi^2 = (1 - r^2) \sum_{i=0}^{N-1} (y_i - \bar{y})^2 \quad (15.2.13)$$

For data with varying errors σ_i , the above equations remain valid if the sums in equations (15.2.13) and (14.5.1) are weighted by $1/\sigma_i^2$.

The following object, `Fitab`, carries out exactly the operations that we have discussed. You call its constructor either with, or without, errors σ_i . If the σ_i 's are known, the calculations exactly correspond to the formulas above. However, when σ_i 's are unavailable, the routine *assumes* equal values of σ for each point and *assumes* a good fit, as discussed in §15.1.

The formulas (15.2.6) are susceptible to roundoff error. Accordingly, we rewrite them as follows: Define

$$t_i = \frac{1}{\sigma_i} \left(x_i - \frac{S_x}{S} \right), \quad i = 0, 1, \dots, N-1 \quad (15.2.14)$$

and

$$S_{tt} = \sum_{i=0}^{N-1} t_i^2 \quad (15.2.15)$$

Then, as you can verify by direct substitution,

$$b = \frac{1}{S_{tt}} \sum_{i=0}^{N-1} \frac{t_i y_i}{\sigma_i} \quad (15.2.16)$$

$$a = \frac{S_y - S_x b}{S} \quad (15.2.17)$$

$$\sigma_a^2 = \frac{1}{S} \left(1 + \frac{S_x^2}{S S_{tt}} \right) \quad (15.2.18)$$

$$\sigma_b^2 = \frac{1}{S_{tt}} \quad (15.2.19)$$

$$\text{Cov}(a, b) = -\frac{S_x}{S S_{tt}} \quad (15.2.20)$$

$$r_{ab} = \frac{\text{Cov}(a, b)}{\sigma_a \sigma_b} \quad (15.2.21)$$

fitab.h

struct Fitab {

Object for fitting a straight line $y = a + bx$ to a set of points (x_i, y_i) , with or without available errors σ_i . Call one of the two constructors to calculate the fit. The answers are then available as the variables a, b, siga, sigb, chi2, and either q or sigdat.

```
    Int ndata;
    Doub a, b, siga, sigb, chi2, q, sigdat;           Answers.
    VecDoub_I &x, &y, &sig;
```

```
    Fitab(VecDoub_I &xx, VecDoub_I &yy, VecDoub_I &ssig)
```

Constructor. Given a set of data points $x[0..ndata-1]$, $y[0..ndata-1]$ with individual standard deviations $\text{sig}[0..ndata-1]$, sets a, b and their respective probable uncertainties siga and sigb, the chi-square chi2, and the goodness-of-fit probability q (that the fit would have χ^2 this large or larger).

```
        Gamma gam;
        Int i;
        Doub ss=0.,sx=0.,sy=0.,st2=0.,t,wt,sxoss;
        b=0.0;                                     Accumulate sums ...
        for (i=0;i<ndata;i++) {
            wt=1.0/SQR(sig[i]);                    ...with weights
            ss += wt;
            sx += x[i]*wt;
            sy += y[i]*wt;
        }
        sxoss=sx/ss;
        for (i=0;i<ndata;i++) {
            t=(x[i]-sxoss)/sig[i];
            st2 += t*t;
            b += t*y[i]/sig[i];
        }
        b /= st2;                                  Solve for a, b,  $\sigma_a$ , and  $\sigma_b$ .
        a=(sy-sx*b)/ss;
        siga=sqrt((1.0+sx*sx/(ss*st2))/ss);
        sigb=sqrt(1.0/st2);                         Calculate  $\chi^2$ .
        for (i=0;i<ndata;i++) chi2 += SQR((y[i]-a-b*x[i])/sig[i]);
        if (ndata>2) q=gam.gammq(0.5*(ndata-2),0.5*chi2);   Equation (15.2.12).
    }
```

```
    Fitab(VecDoub_I &xx, VecDoub_I &yy)
```

Constructor. As above, but without known errors (sig is not used). The uncertainties siga and sigb are estimated by assuming equal errors for all points, and that a straight line is a good fit. q is returned as 1.0, the normalization of chi2 is to unit standard deviation on all points, and sigdat is set to the estimated error of each point.

```
        Int i;
        Doub ss,sx=0.,sy=0.,st2=0.,t,sxoss;
        b=0.0;                                     Accumulate sums ...
        for (i=0;i<ndata;i++) {
            sx += x[i];                             ...without weights.
            sy += y[i];
        }
        ss=ndata;
        sxoss=sx/ss;
        for (i=0;i<ndata;i++) {
            t=x[i]-sxoss;
            st2 += t*t;
            b += t*y[i];
        }
        b /= st2;                                  Solve for a, b,  $\sigma_a$ , and  $\sigma_b$ .
        a=(sy-sx*b)/ss;
        siga=sqrt((1.0+sx*sx/(ss*st2))/ss);
        sigb=sqrt(1.0/st2);                         Calculate  $\chi^2$ .
        for (i=0;i<ndata;i++) chi2 += SQR(y[i]-a-b*x[i]);
```